Figure 9.15 Design option 2 for the Cast data product

To learn something new, we won't repeat the pro-con-fix analysis, and propose the tradeoff analysis instead.

KEY POINT Tradeoffs are needed everywhere. The Oxford English Dictionary defines *tradeoff* as to "exchange something of value, especially as part of a compromise." It is a fundamental concept in architecture design. Whenever we choose architecture options, one is rarely utterly superior to another. Usually, we exchange some quality attributes or values in favor of others. As an architect, you need to be aware that you lose something to gain something else with every decision.

Let's think about what we are exchanging in these two designs. A simple example of a tradeoff in our case is an exchange of loose coupling (event contract in option 1) for rapid development and platform reuse (Airflow workflow in option 2). When you establish what you are exchanging, the next step is to think about what we and our product owners value more. You should look for hints in functional and nonfunctional requirements. If it's not there and you cannot answer this question, you should explicitly ask your stakeholders.

We already resolved one of the data products that has its source in the Hitchcock Movie Maker system, so now let's look at another data product with the same source.

SCRIPTS DATA PRODUCT

Here's a summary of architectural drivers for the Scripts data product:

- The script recommender will consume Scripts data product through a REST API. The script Recommender does not require any extraordinary transformation of scripts.
- We discovered the following quality attribute:
  - *Security*—Scripts are essential for Messflix. This is why they should be protected.
- The script recommender is not a real-time application. It will consume data no more than once a day. Scalability, availability, and performance are not problems.